

Submission Worksheet

Submission Data

Course: IT202-007-F2025

Assignment: IT202 Milestone 1

Student: Roshan T. (rt524)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 11/10/2025 11:00:33 PM

Updated: 11/10/2025 11:00:33 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-1/grading/rt524>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-007-F2025/it202-milestone-1/view/rt524>

Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>
- 1. Refer to Milestone1 of this doc:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone1 branch
 - 1. `git checkout Milestone1` (ensure proper starting branch)
 - 2. `git pull origin Milestone1` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone1`
 - 4. On Github merge the pull request from Milestone1 to dev
 - 5. On Github create a pull request from dev to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
- 10. `git checkout dev`
- 11. `git pull origin dev`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

≡ Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

```
<?php
require($_DIR__."/../partials/nav.php");
//
<h2>Register</h2>
<form onsubmit="return validate(this)" method="POST">
  <div>
    <label for="email">Email</label>
    <input id="email" type="email" name="email" required />
  </div>
  <div>
    <label for="username">Username</label>
    <input type="text" name="username" required minlength="4" />
  </div>
  <div>
    <label for="pw">Passwords</label>
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <div>
    <label for="confirm">confirm</label>
    <input type="password" name="confirm" required minlength="8" />
  </div>
  <input type="submit" value="Register" />
</form>
```

HTML validations register

Register

Email



Please include an '@' in the email address. 'r' is missing an '@'.

Confirm

Register

email validation

Register

Email
Username
Password

⚠ Please lengthen this text to 8 characters or more (you are currently using 1 character).

password validation

📄 Saved: 11/10/2025 9:21:57 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validation here is the length of the passwords that must match the minimum requirement, which is 8 characters. This is done through minlength="8" in the label for the corresponding field.

📄 Saved: 11/10/2025 9:21:57 PM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

📄 Part 1:

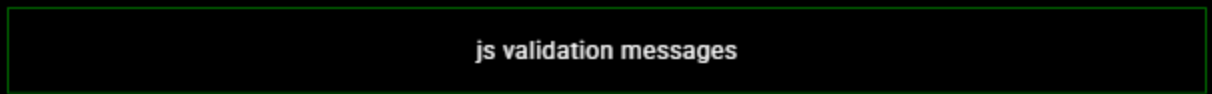
Progress: 100%

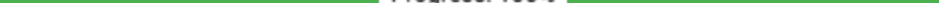
Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

📄 Saved: 11/10/2025 9:21:57 PM

javascript validate() function



Part 2:  Progress: 100%

Your Response:

The function calls check the length being 8 characters as well as the validity of the email format, which is checked through regex. The username is also checked through regex and the message is displayed.

Task #3 (0.33 pts.) - PHP Validation (steps before the DB

Progress: 100%

 **Part 1:**

- Show the code related to the register page's PHP validation

- Show the code related to the register page & PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

```

<?php
// PHP REGISTER PAGE
// 1. GET THE INPUTS
$username = $_POST['username'];
$password = $_POST['password'];
$confirm = $_POST['confirm'];
$email = $_POST['email'];

// 2. VALIDATE THE INPUTS
// 2.1 Check if email is empty
if (empty($email)) {
    $message = "Email must not be empty.";
    $message .= "Register";
}

// 2.2 Check if password is empty
if (empty($password)) {
    $message = "Password must not be empty.";
    $message .= "Register";
}

// 2.3 Check if confirm is empty
if (empty($confirm)) {
    $message = "Confirm password must not be empty.";
    $message .= "Register";
}

// 2.4 Check if password and confirm match
if ($password != $confirm) {
    $message = "Password and confirm must match.";
    $message .= "Register";
}

// 2.5 Check if email is valid
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $message = "Invalid email address.";
    $message .= "Register";
}

// 2.6 Check if username is valid
if (!preg_match("/^[a-zA-Z0-9_]{3,20}$/", $username)) {
    $message = "Username must be lowercase, alphanumerical, and can only contain _ or -";
    $message .= "Register";
}

// 2.7 Check if password is long enough
if (strlen($password) < 8) {
    $message = "Password must be at least 8 characters long.";
    $message .= "Register";
}

// 2.8 Check if passwords match
if ($password != $confirm) {
    $message = "Passwords must match.";
    $message .= "Register";
}

// 3. If there is a message, show it
if ($message) {
    echo $message;
} else {
    // 4. If no message, register the user
    // 4.1 Connect to the database
    $db = new mysqli("localhost", "root", "", "mydb");

    // 4.2 Check if the email is already in the database
    $sql = "SELECT * FROM users WHERE email = ?";
    $result = $db->query($sql);

    if ($result->num_rows > 0) {
        $message = "Email already in use.";
        $message .= "Register";
    } else {
        // 4.3 Check if the username is already in the database
        $sql = "SELECT * FROM users WHERE username = ?";
        $result = $db->query($sql);

        if ($result->num_rows > 0) {
            $message = "Username already in use.";
            $message .= "Register";
        } else {
            // 4.4 Insert the user into the database
            $sql = "INSERT INTO users (username, password, email) VALUES (?, ?, ?)";
            $result = $db->query($sql);

            if ($result) {
                // 4.5 Redirect to the login page
                header("Location: login.php");
                exit();
            } else {
                $message = "Registration failed. Please try again later.";
                $message .= "Register";
            }
        }
    }
}

// 5. If there is no message, show the success message
if (!$message) {
    $message = "Registration successful. Please check your email for confirmation.";
    $message .= "Register";
}

// 6. Show the message
echo $message;

```

php validation

Invalid email address.
Register

Email

Username

Password

Confirm

Register

php invalid email

Email must not be empty.
Register

Invalid email address.
Email

Username must be lowercase, alphanumerical, and can only contain _ or -
Username

Password must not be empty.
Password

Confirm password must not be empty.
Confirm

Register

Confirm password must not be empty.

Password must be at least 8 characters long.

Passwords must match.

php validations for register

 Saved: 11/10/2025 9:30:31 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the

requirement)

Your Response:

The fields of the form are checked if they are empty using empty(string) and if this returns true, provides a message. The email is first sanitized to ensure no injections can occur and then checked if it is valid using the is_valid_email function, the same being done to the password.



Saved: 11/10/2025 9:30:31 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/rt-524/rt524-it2023007/Milestone1/public_html/project/register.php



URL

<https://github.com/rt-524/rt5>



URL #2

<https://rt524-it202-007-dev-5d0617ee62f2.herokuapp.com/project/register.php>



URL

<https://rt524-it202-007-dev-5d0617ee62f2.herokuapp.com/project/register.php>



URL #3

<https://github.com/rt-524/rt524-it2023007>



URL

<https://github.com/rt-524/rt5>



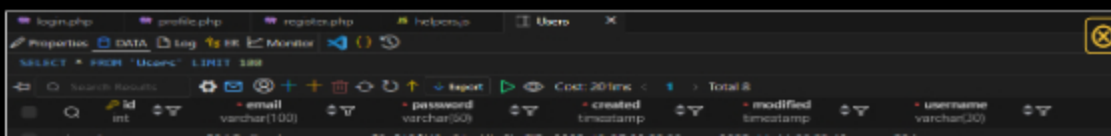
Saved: 11/10/2025 9:42:03 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



> 1	rs246@rs2edu	\$2y\$12\$H0uN6dAILv9qzP	2025-11-27 03:22:39	2025-11-11 00:23:42	rs24
> 2	testinguser@test.org	\$2y\$12\$1NesM6w7TvAGP3	2025-11-10 16:03:47	2025-11-10 16:03:47	testuser
> 3	testing1@test.com	\$2y\$12\$13a7sfIM5fEmqg	2025-11-10 16:04:49	2025-11-10 16:04:49	andfasdfadaf
> 6	testuser@gmail.com	\$2y\$12\$a2bCf17nLq6V00%	2025-11-10 23:40:45	2025-11-10 23:40:45	testuser1
> 7	shortuser@short.com	\$2y\$12\$ad7gequ6yU01R0t	2025-11-11 00:36:51	2025-11-11 00:36:51	12312
> 8	numUser@num.com	\$2y\$12\$1N6A8ZVzP7VgPV	2025-11-11 00:37:21	2025-11-11 00:37:21	10239
> 9	zindag@zindag.com	\$2y\$12\$Q6HqAyyF7NheRC	2025-11-11 00:38:18	2025-11-11 00:38:18	10298302893
> 13	13td@13td.com	\$2y\$12\$9H4My8lv560U8en	2025-11-11 01:12:32	2025-11-11 01:12:32	over30charactersover30cha

vscode database registered users

 Saved: 11/10/2025 9:42:46 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

≡ Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

```
<?php
require(__DIR__ . '/../partials/nav.php');
?>
<h3>Login</h3>
<form onsubmit="return validate(this)" method="POST">
  <div>
```



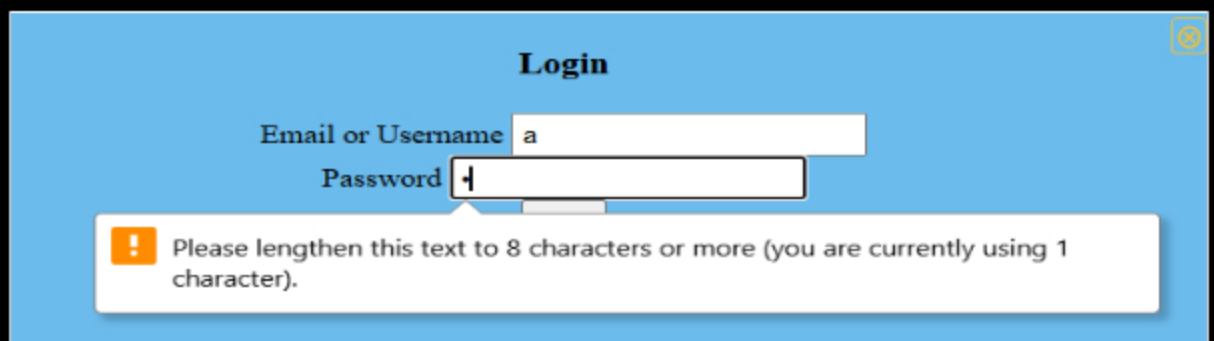
```

<label for="email">Email or Username</label>
<input id="email" type="text" name="email" required />
</div>
<div>
<label for="pw">Password</label>
<input type="password" id="pw" name="password" required minlength="8" />
</div>
<input type="submit" value="Login" />
</form>

```

html login form

html empty username



html short password

 Saved: 11/10/2025 9:47:54 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

required length is 8 characters and the password is not, returns the length of the input and the required input

 Saved: 11/10/2025 9:47:54 PM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Progress: 100%

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



Password must be at least 8 characters

Email or username is invalid

Password

Login

js validation messages

js validation code snippet



Saved: 11/10/2025 11:00:33 PM

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

uses `isValidEmail()` && `isValidUsername()` with the same input to check if it is not one or the other using regex comparisons, then returns the error message. uses the same password checking as register with `isValidPassword` to check length.



Saved: 11/10/2025 11:00:33 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch

```
1 <?php  
2 // ...  
3  
4 // Validation functions  
5  
6 function validateEmail($email) {  
7     return filter_var($email, FILTER_VALIDATE_EMAIL);  
8 }  
9  
10 function validateUsername($username) {  
11     return preg_match('/^[a-z0-9_-]{3,16}$/', $username);  
12 }  
13  
14 function validatePassword($password) {  
15     return preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@_!#$%^&*()~`{|}~']).{8,}$/', $password);  
16 }  
17  
18 // Login logic  
19  
20 if ($_POST['login']) {  
21     $email = $_POST['email'];  
22     $password = $_POST['password'];  
23  
24     if (!validateEmail($email) || !validateUsername($_POST['username'])) {  
25         echo "Email/Username must not be empty.  
26         Username must be lowercase, alphanumerical, and can only contain - or _";  
27     }  
28     if (!validatePassword($password)) {  
29         echo "Password must not be empty.  
30         Password must be at least 8 characters long.  
31     }  
32 }  
33  
34 // ...  
35
```

php validations login

Login Register

Email/Username must not be empty.
Username must be lowercase, alphanumerical, and can only contain - or _
Password must not be empty.
Password must be at least 8 characters long.

php error messages

Invalid email address.

Login

Password must be at least 8 characters long.

Email or Username
Password

invalid email address php validation



Saved: 11/10/2025 9:58:39 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

first checks if all fields are empty and provides error messages based on that. Then checks the sanitized email to ensure that it is in a valid format and prevent any possible injections. Otherwise can be treated as a username and checked to see if it follows the valid username format, and checks the validity of the password's length.



Saved: 11/10/2025 9:58:39 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/rt-524/rt524-it202-007/Milestone1/public_html/project/login.php



URL

<https://github.com/rt-524/rt5>

URL #2

<https://rt524-it202-007-dev-5d0617ee62f2.herokuapp.com/project/login.php>



URL

<https://rt524-it202-007-dev-5>

URL #3

<https://github.com/rt-524/rt524-it202-00711>



URL

<https://github.com/rt-524/rt5>

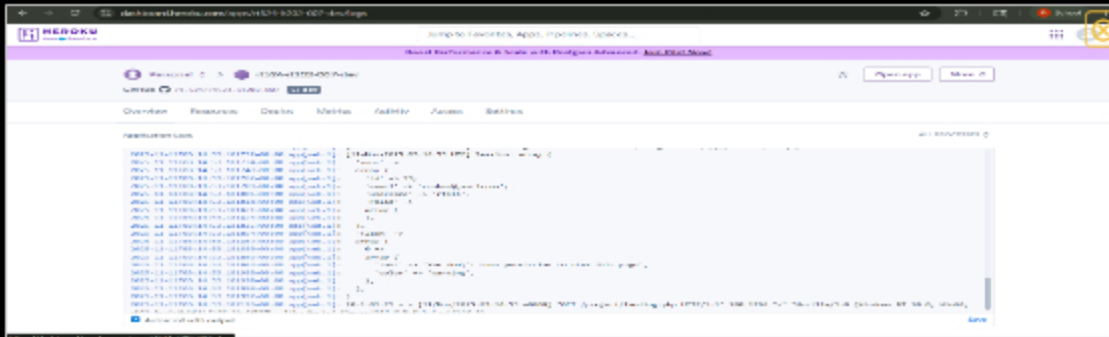
Saved: 11/10/2025 10:00:00 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

- From the heroku logs (Dashboard -> More button -> view Logs), cap!
- It should show the id, username, email, and roles

A screenshot of the Heroku dashboard showing the logs for an application. The logs display a series of log entries, including a successful login message: "User logged in successfully. User ID: 1, Username: admin, Email: admin@example.com, Roles: [\"admin\"]". The logs are displayed in a table format with columns for time, log level, and log message.

heroku error log output

Saved: 11/10/2025 10:15:34 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

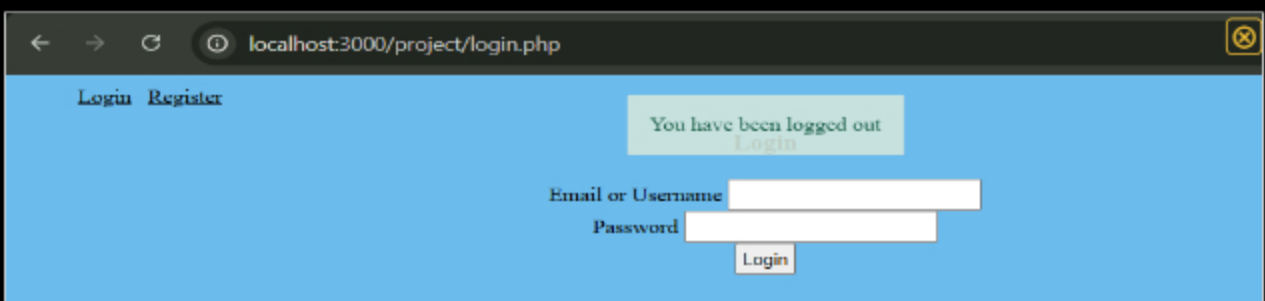
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message
- Show the code snippet of the logout page

A screenshot of a web application interface. At the top, there is a navigation bar with "Login" and "Register" links. Below the navigation bar, there is a message box that says "You have been logged out" with a "Login" button. Below the message box, there is a login form with fields for "Email or Username" and "Password", and a "Login" button.

logout message

public_html > project >  logout.php

You, 5 hours ago | 1 author (You)

```
1 <?php
2 // require functions.php to pull in flash()
3 require(__DIR__ . "/../../lib/functions.php");
4 reset_session(); // clear session data and start a new session
5 flash("You have been logged out","success");
6 header("Location: $BASE_PATH/login.php"); // redirect back to login
```

You, 5 hours ago

logout code snippet



Saved: 11/10/2025 10:06:03 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/rt-524/rt524-it202300771>



URL

<https://github.com/rt-524/rt524-it>



Saved: 11/10/2025 10:06:03 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The session is first cleared entirely using `reset_session()` and a flash message is called to notify the user that they have been logged out. The flash function was included through the `require` function. After pressing the logout button, they are then redirected to the login page using `BASE_PATH`



Saved: 11/10/2025 10:06:03 PM

Section #4: (2 pts.) Feature: Security Rules

Task #1 (1 pt.) - Database Tables

Progress: 100%

Details:

- From the vs code mysql tool, show both the Roles and UserRoles tables

The screenshot shows the MySQL tool interface in VS Code. The left sidebar displays the database schema with 'Roles' selected. The main pane shows the 'Roles' table structure and data.

id	name	description	is_active	created_timestamp	modified_timestamp
1	Admin		1	2025-11-10 22:20:43	2025-11-10 22:20:43
2	New Role	This is a new test role	1	2025-11-10 22:45:30	2025-11-10 22:45:30

Roles table

The screenshot shows the MySQL tool interface in VS Code. The left sidebar displays the database schema with 'UserRoles' selected. The main pane shows the 'UserRoles' table structure and data.

user_id	role_id	is_active	created_timestamp	modified_timestamp
1	1	1	2025-11-10 22:29:25	2025-11-10 22:29:25
2	1	0	2025-11-10 22:50:21	2025-11-10 22:50:21

user roles table

Saved: 11/10/2025 10:06:57 PM

Task #2 (1 pt.) - Demo Security Rules

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the message related to needing to be logged in (i.e., try to manually
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function

You don't have permission to view this page

Login

You must be logged in to view this page

Email or Username

Password

Login

messages for needing to be logged in and proper perms

```
function is_logged_in($redirect = false, $destination = "login.php")
{
    $isLoggedIn = isset($_SESSION["user"]);
    if ($redirect && !$isLoggedIn) {
        //if this triggers, the calling script won't receive a reply since die()/exit() terminates it
        flash("You must be logged in to view this page", "warning");
        $path = get_url($destination);

        die(header("Location: $path"));
    }
    return $isLoggedIn;
}
```

login check function

```
if (!has_role("Admin")) {
    flash("You don't have permission to view this page", "warning");
    die(header("Location: " . get_url("landing.php")));
}
```

role check admin function

```
20 function has_role($role)
21 {
22     if (is_logged_in() && isset($_SESSION["user"]["roles"])) {
23         foreach ($_SESSION["user"]["roles"] as $r) {
24             if ($r["name"] === $role) {
25                 return true;
26             }
27         }
28     }
29     return false;
30 }
```

role check function



Saved: 11/10/2025 10:19:32 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/rt-524/rt524-it202300718>



URL

<https://github.com/rt-524/rt524-it>



Saved: 11/10/2025 10:19:32 PM

⇒ Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

the login check first checks to see if you are logged in by seeing if the session for a user is set or not. If the session is not set, returns an error message and redirects you back. Otherwise, allows you to continue.

The role check code works by first checking if the user is logged in and if the role is set for a user. It then iterates through the users roles to get their names, otherwise it returns false if not applicable.



Saved: 11/10/2025 10:19:32 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

≡ Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)

```
<div class="mb-3">
  <input type="email" name="email" id="email" value="{<php echo($email); }"/>
</div>
<div class="mb-3">
  <input type="text" name="username" id="username" value="{<php echo($username); }"/>
</div>
<div class="mb-3">
  <input type="password" name="currentPassword" id="cp" />
</div>
<div class="mb-3">
  <input type="password" name="newPassword" id="np" />
</div>
<div class="mb-3">
  <input type="password" name="confirmPassword" id="cmp" />
</div>
<input type="submit" value="Update Profile" name="save" />
</form>
```

html code for profile

Profile

Email

Current Password

New Password

Confirm Password

Update Profile

Please enter a part following '@'. 'roshan@' is incomplete.

email @ error message

Saved: 11/10/2025 10:24:33 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

New Password
Confirm Password
Update Profile

passwords dont match



Saved: 11/10/2025 10:58:27 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code first checks the length of the password with the `isValidPassword(pw)` function. Then, individually checks the validity of the email and the username with their respective JS functions, checked with regex for validity.



Saved: 11/10/2025 10:58:27 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions

```
10 <script>
11 // Validation functions
12 function isValidEmail(email) {
13     const emailRegex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
14     return emailRegex.test(email);
15 }
16 function isValidUsername(username) {
17     const usernameRegex = /^[a-zA-Z0-9_]{3,20}$/;
18     return usernameRegex.test(username);
19 }
20 function isValidPassword(password) {
21     const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*^&])[a-zA-Z\d@$!%*^&]{8,}$/;
22     return passwordRegex.test(password);
23 }
24 function validateForm() {
25     const username = document.getElementById('username').value;
26     const email = document.getElementById('email').value;
27     const password = document.getElementById('password').value;
28     const confirmPassword = document.getElementById('confirm_password').value;
29
30     if (!isValidEmail(email)) {
31         document.getElementById('email_error').innerHTML = 'Invalid email format';
32         return false;
33     }
34     if (!isValidUsername(username)) {
35         document.getElementById('username_error').innerHTML = 'Invalid username format';
36         return false;
37     }
38     if (!isValidPassword(password)) {
39         document.getElementById('password_error').innerHTML = 'Invalid password format';
40         return false;
41     }
42     if (password !== confirmPassword) {
43         document.getElementById('confirm_password_error').innerHTML = 'Passwords do not match';
44         return false;
45     }
46
47     // If all validations pass, the form is valid
48     return true;
49 }
50
51 // Example of how to use the validation functions
52 document.getElementById('username').addEventListener('input', function() {
53     if (!isValidUsername(this.value)) {
54         document.getElementById('username_error').innerHTML = 'Invalid username format';
55     }
56 });
57
58 document.getElementById('email').addEventListener('input', function() {
59     if (!isValidEmail(this.value)) {
60         document.getElementById('email_error').innerHTML = 'Invalid email format';
61     }
62 });
63
64 document.getElementById('password').addEventListener('input', function() {
65     if (!isValidPassword(this.value)) {
66         document.getElementById('password_error').innerHTML = 'Invalid password format';
67     }
68 });
69
70 document.getElementById('confirm_password').addEventListener('input', function() {
71     if (password !== this.value) {
72         document.getElementById('confirm_password_error').innerHTML = 'Passwords do not match';
73     }
74 });
75
76 // Submit button click event
77 document.getElementById('submit').addEventListener('click', function() {
78     if (validateForm()) {
79         // Form is valid, proceed with the database call
80         document.getElementById('submit').disabled = true;
81     }
82 });
```


[illegible]



Saved: 11/10/2025 10:46:32 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/rt-524/rt524-it202901720>

URL

<https://github.com/rt-524/rt524-it>

Saved: 11/10/2025 10:46:32 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Projects • rt524-it202-007

4 hrs 23 mins over the Last 7 Days in rt524-it202-007 under all branches. 📄

wakatime overall

Files

1 hr 36 mins public_html/project/login.php
 84 mins public_html/project/register.php
 32 mins public_html/project/profile.php
 28 mins public_html/project/assign_roles.php
 15 mins lib/functions.php
 11 mins lib/user_helpers.php
 6 mins public_html/project/styles.css
 5 mins C:\html\project\admin\assign_roles.php
 5 mins public_html/project/logout.php
 5 mins public_html/project/admin/list_roles.php
 5 mins C:\project\sql\user_insert_admin_role.sql
 4 mins C:\html\project\admin\create_role.php
 4 mins lib/get_url.php
 2 mins C:\project\sql\002_alter_table_users.sql
 1 min C:\sql\003_create_table_users.sql
 1 min public_html/project/index.php
 1 min lib/user_sessions.php
 1 min public_html/index.php
 1 min lib/validation.php
 1 min C:\project\sql\001_create_table_roles.sql
 60 mins public_html/project/details.php
 57 mins lib/duplicate_user_details.php

Branches

1 hr 27 mins Master
 1 hr 10 mins Fast-MST-UserLoginEnhancement
 52 mins Fast-MST-UserRoles
 27 mins Fast-MST-UserProfile
 24 mins develop-MST-dying-out



Saved: 11/10/2025 10:48:12 PM

≡ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

Throughout this milestone, I learned the necessity of validation in every part of a form. This can be through HTML, JavaScript, and PHP which all provide different layers of safety. HTML and JavaScript ensure that the initial values of the form submit with the proper values. The PHP side of validation checks the validity in terms of the database, first sanitizing everything to prevent SQL injections and then providing an additional layer of validation before putting anything into the database that the client side validations may have missed.



Saved: 11/10/2025 10:50:30 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this assignment was going through the SQL queries and dealing with the databases as I am most familiar with this as I'm taking CS331 at the same time. This allowed me to further understand how the database works on a deeper level as well as how most sides would go through their validations and database setup in terms of registration, logins, and updates for profiles.



Saved: 11/10/2025 10:51:30 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this assignment for me was keeping track of all the different functions and when I should be using repeats of things, making functions of repeated things, or how certain functions are accessed. Sometimes, I did not fully understand why a new file or function needed to be made, but once we started to add more functionality and those functions or files were used more, I saw that they were necessary when scaling our project more.



Saved: 11/10/2025 10:52:43 PM